

Task 3 — Performance Evaluation with Amdahl's and Gustafson's Laws

FIT3143 Lab #2. Supporting notes for slide section (c). All numbers come from [bench/analysis-caas/summary.csv](#), produced by [bench/analyse.py](#) from the raw CSVs in [bench/results-caas/](#).

How the measurements were taken

Machine.

- Monash CAAS, partition [defq](#): AMD EPYC 7763 nodes, 16 cores each, joined by **Gigabit ethernet** (not InfiniBand). The network turns out to be the limit at high rank counts.
- Every figure is measured across **4 nodes / 64 cores**, so worker counts above 16 run on separate physical machines, not oversubscribed onto one.
- Each configuration is run 3 times and reduced by **median**. CAAS nodes may be shared with other jobs; a median ignores one disturbed run, a mean would not.

Problem sizes.

- The n -sweep covers **30 distinct values** from 20,000,000 to 100,000,000, spaced geometrically. Serial runs 5.49 s at $n = 20\text{M}$ and 53.72 s at $n = 100\text{M}$.
- The spec warns against runtimes under 1 s. 43 of 207 measured configurations cross that line — all high-worker-count runs at $n \leq 54\text{M}$, the quickest being the hybrid at 4×16 finishing $n = 20\text{M}$ in **0.33 s**. Everything at $n \geq 57\text{M}$ is above it.
- The effect is visible: across the lowest eight n , the hybrid's speedup scatters 14.4% between neighbouring points that differ in n by only 6%. That is noise, not the algorithm.
- **Every headline figure is therefore quoted at $n = 100\text{M}$** , where the fastest run still takes 1.76 s and the ranking is stable. The low- n points are kept because the spec asks for 30 values and the *trend* is sound; individual points there are not precise.

Denominator. Every speedup is against the **Week 4 Task 1 serial program at the same n** , re-measured on CAAS, as the spec requires — never against the parallel program's own one-worker run.

Partitioning scheme.

- Headline figures use [blockcyclic](#) with a chunk of **4096 candidates** (the default everywhere). It interleaves like [cyclic](#) but hands out 4096 neighbouring candidates at a time, which keeps cache locality.
- The master-worker [dynamic](#) scheme appears only in the scheme comparison (§3). It reserves rank 0 as a pure dispatcher, which costs a worker and breaks the spec's rule that *every process*, including the root, does a share of the work.

1. Why the two laws need different experiments

The spec's warning is the crux: *"these two laws have different measurement assumptions, and you will need to design measurement experiments to compute the serial/parallel fractions correctly."* They cannot

share one measured fraction.

	Amdahl	Gustafson
Held fixed	problem size n	execution time
Question	how much faster is one fixed workload?	how much more work fits in the same time?
Formula	$S(W) = 1 / (s + p/W)$	$S(W) = s + p \cdot W$
s is the serial share of what	the one-worker execution	the parallel execution as actually run
So s is measured on	the serial program at that n	the $P \times T$ run being scored

Feeding Amdahl a fraction from a P -way run is circular. As workers are added the parallel part shrinks but the serial part does not, so the measured serial share of a parallel run rises with W — in our data from **0.015 at $W=1$ to 0.42 at $W=64$** . Put that into Amdahl and the "ceiling" sags as W grows, which is no ceiling at all. So two experiments are run.

Experiment A — Amdahl's fraction, from the serial program

The serial program ([week-4-lab-1/task1.c](#)) has four phase timers covering its whole runtime:

<code>t_alloc</code>	allocate the result buffer	serial
<code>t_compute</code>	the trial-division search	PARALLEL (this is p)
<code>t_merge</code>	assemble the ordered result	serial
<code>t_io</code>	write the output file	serial
<code>t_total</code>	whole run	

```
s = median(t_alloc + t_merge + t_io) / (that + median(t_compute))
p = 1 - s
```

This depends only on n , which is why the Amdahl curve in figures 6 and 7 is the same for MPI and hybrid: both parallelise the same phase of the same algorithm.

n	s	p	ceiling $S(\infty) = 1/s$
20,000,000	0.0212	0.9788	47.1×
45,979,000	0.0149	0.9851	67.0×
100,000,000	0.0090	0.9910	110.9×

The serial fraction falls as n grows. Trial division costs $\sim O(n^{1.5}/\log n)$ while allocation, ordering and I/O scale with $\pi(n) \approx n/\log n$, so the parallel part outgrows the serial part and the ceiling rises from 47.1×

to 110.9×. Bigger n does not make the program faster; it raises the limit on how much faster parallelism could make it.

Experiment B — Gustafson's fraction, from each parallel run

Gustafson's s is by definition the serial share of the parallel run, so it is measured on the run being scored. Both parallel programs split their runtime the same way:

<code>t_alloc</code>	buffer allocation	serial
<code>t_bcast</code>	MPI_Bcast of n to every rank	serial
<code>t_compute</code>	the distributed search (MPI_MAX over ranks)	PARALLEL
<code>t_lsort</code>	each rank ordering its own results [Task 2]	PARALLEL
<code>t_comm</code>	MPI_Gather + MPI_Gatherv of results	serial
<code>t_sort</code>	root ordering the gathered result	serial
<code>t_io</code>	root writing the output file	serial
<code>t_total</code>	MPI_Init to just before MPI_Finalize	

Two measurement decisions:

- **`t_compute` is reduced with MPI_MAX, not averaged.** A phase ends when its slowest rank ends; an average would hide load imbalance. **MPI_MIN** is reduced alongside and the ratio reported as **imbalance**.
- **`t_lsort` counts as parallel.** Every rank runs it simultaneously on its own data. Counting it as serial would inflate s for the hybrid only.

A barrier precedes the compute timer so it measures the phase, not rank skew.

The cross-check — Karp–Flatt

Neither law charges for communication, so both over-predict. To find *which* cause explains the gap, Amdahl is inverted on the **measured** speedup:

$$e = (1/S - 1/W) / (1 - 1/W) \quad (\text{undefined at } W = 1)$$

- Flat e as W grows → the gap is genuine serial work.
- Rising e → the loss is **parallel overhead** that grows with W (broadcast, gather, bandwidth, oversubscription), which neither law models.

2. Results (4 nodes × 16 cores)

Empirical speedup against problem size

Sweep A holds the configuration fixed (Task 1 at 16×1, Task 2 at 4×16) and walks n across all 30 values, each against serial at the same n .

n	serial	MPI 16×1	vs its Amdahl	hybrid 4×16	vs its Amdahl	OpenMP 1×16	Pthreads 1×16
20,000,000	5.49 s	9.40×	77%	16.85×	62%	11.52×	9.34×

n	serial	MPI 16×1	vs its Amdahl	hybrid 4×16	vs its Amdahl	OpenMP 1×16	Pthreads 1×16
29,494,000	9.52 s	10.33×	84%	18.79×	66%	11.46×	9.06×
43,497,000	16.48 s	11.04×	84%	21.04×	63%	12.39×	9.71×
67,808,000	30.93 s	12.16×	89%	24.41×	65%	12.45×	9.93×
100,000,000	53.72 s	12.30×	87%	30.59×	75%	13.51×	10.35×

Every implementation gets faster relative to serial as n grows, and the MPI ones gain most: Task 1 9.40× → 12.30× (+31%), Task 2 16.85× → 30.59× (+82%), against OpenMP +17% and Pthreads +11%. Two mechanisms compound:

- The serial fraction falls from 0.0212 to 0.0090, so the Amdahl ceiling rises (§1). This helps every implementation.
- The gather is a fixed cost set by $\pi(n)$ while the parallel part grows as $n^{1.5}/\log n$, so communication shrinks as a *share* of the run. This is why Task 1 closes from 77% of its bound to 87% and Task 2 from 62% to 75%. **The shared-memory baselines have no such term and do not close.**

Message passing is punished at small n and rewarded at large n — the strongest argument in the study for using MPI on this problem.

Task 1 — Open MPI, increasing rank count

$n = 100\text{M}$, where $s = 0.0090$ gives an Amdahl ceiling of 110.9×.

W (ranks)	measured	Amdahl	Gustafson	Karp–Flatt e	efficiency
1	1.00×	1.00×	1.00×	—	100%
2	1.96×	1.98×	1.97×	0.0229	98%
4	3.76×	3.89×	3.81×	0.0211	94%
8	7.00×	7.53×	7.21×	0.0205	87%
12	9.89×	10.92×	10.20×	0.0194	82%
16	12.26×	14.09×	12.54×	0.0203	77%
24	16.82×	19.88×	17.34×	0.0185	70%
32	20.88×	25.01×	21.33×	0.0172	65%
48	23.34×	33.72×	27.99×	0.0225	49%
64	23.87×	40.82×	37.49×	0.0267	37%

Karp–Flatt is flat at $e \approx 0.018\text{--}0.023$ from $W=2$ to $W=32$, then rises at 48 and 64. This is the central diagnostic result.

- Flat e to $W=32$ means the gap below Amdahl is genuine serial work; the program scales as well as its serial fraction allows.
- The rise at $W \geq 48$ ($0.0172 \rightarrow 0.0225 \rightarrow 0.0267$) is where communication starts to cost more than it saves: 48 ranks buy $23.34\times$, 64 buy only $23.87\times$ — 2% more speedup for 33% more hardware.
- Even the flat e (≈ 0.020) sits $2.2\times$ above the true serial fraction (0.0090). That residual is the gather, which the serial program never does.

What the shortfall is made of

Amdahl predicts $40.82\times$ at $W=64$; we measure $23.87\times$. Against 53.72 s serial that is 1.32 s predicted vs 2.25 s measured — **0.93 s** to explain. Three costs, and **none of them exist in the program Amdahl's s was measured on:**

1. The gather is a fixed cost that does not shrink with W . Every prime must reach rank 0 once, so volume is set by $\pi(n)$, not rank count:

```
 $\pi(1e8) = 5,761,455 \text{ primes} \times 8 \text{ bytes} = 46.1 \text{ MB}$ 
 $46.1 \text{ MB} / 125 \text{ MB/s (1 Gb/s)} = 0.369 \text{ s floor, before latency}$ 
```

Compute per rank falls as $1/W$; this does not fall at all, so its *share* of the run grows with every rank. 0.369 s alone is ~40% of the 0.93 s shortfall.

2. The root's global sort is a serial phase the baseline does not have. `blockcyclic` interleaves chunks, so the gathered result arrives unordered and the root must `qsort` 5,761,455 longs. The serial program appends in ascending order by construction (its `t_merge` is exactly zero), so this phase is invisible to Amdahl's s . Its cost is the `t_sort` column of `bench/results-caas/mpi-*.csv`.

3. Message latency scales with rank count. `MPI_Gatherv` at 64 ranks is 64 transfers to one receiver; the per-message cost is paid 64 times regardless of volume.

One negative result: there is **no jump at the node boundary**. Ranks 1–16 fit on one node and 24+ span several, yet efficiency falls smoothly through the crossing (76.6% at $W=16$, 70.1% at $W=24$) and e even dips ($0.0203 \rightarrow 0.0185$). The network shows up only once compute has shrunk enough for a fixed ~0.37 s gather to matter — at $W \geq 48$, exactly where e turns up. That is what the fixed-cost model predicts.

In short: both laws model how work is *divided*; neither models how it is *collected back*. Karp–Flatt absorbs the gather and sort into its one serial term, which is why it reports $2.2\times$ more "serial work" than the algorithm has. Prime search is unusually hostile to message passing — 46 MB of output for a parallel phase of ~2 s. A workload returning a single scalar would pay none of costs 1 or 2.

Task 2 — hybrid, rank $\times$ thread trade-off at fixed worker count

total W	best split	speedup	same program at P $\times$ 1	hybrid advantage
16	8 $\times$ 2	12.78 $\times$	12.66 $\times$	+1%

total W	best split	speedup	same program at P×1	hybrid advantage
32	2×16	21.11×	18.65×	+13%
64	8×8	30.37×	22.52×	+35%

- The fourth column is `task2.c` with **one thread per rank**, not `task1.c` — the fair control, because it holds the program constant. It reads 22.52× at 64×1 where Task 1 reads 23.87× because the hybrid still pays `omp` team setup and `t_lsrt` with a single thread; that ~6% is the standing cost of the hybrid structure.
- At W=64 the full ordering is 8×8 (30.37×), 4×16 (30.01×), 16×4 (29.70×), 32×2 (24.78×), 64×1 (22.52×). **Every split with ≥8 threads per rank beats every split with ≤2.**
- A thread shares its rank's memory, so work split across threads adds no gather traffic; work split across ranks does. At W=16 there is little gather to avoid; at W=64 avoiding it is worth a third of the runtime.

This is the spec's matched-worker-count comparison, and it favours the hybrid only once the rank count is high enough for communication to matter.

Threads per MPI process, at a fixed rank count

Rank count held at **P = 4**, only the OpenMP team size varies, n = 100M:

T (threads/rank)	total W	measured	Amdahl	Gustafson	Karp–Flatt e	efficiency
1	4	3.79×	3.89×	3.84×	0.0186	95%
2	8	7.06×	7.53×	7.29×	0.0190	88%
4	16	12.64×	14.09×	13.47×	0.0177	79%
8	32	20.98×	25.01×	22.67×	0.0169	66%
16	64	30.01×	40.82×	36.75×	0.0180	47%

Karp–Flatt is flat across the whole thread axis — 0.0169 to 0.0190, no trend. On the rank axis it climbed to 0.0225 at 48 and 0.0267 at 64. Adding threads accumulates no overhead; adding ranks does, because a thread adds nothing to the 46 MB gather or the message count. That is the whole case for the hybrid.

The same holds at other rank counts, so P = 4 is not a lucky choice:

fixed P	T range	speedup range	e range
4	1 → 16	3.79× → 30.01×	0.0169–0.0190
8	2 → 8	12.78× → 30.37×	0.0168–0.0189
16	1 → 4	12.66× → 29.70×	0.0176–0.0183

The three routes to W = 64 — 4×16, 8×8, 16×4 — land within 2% of each other (30.01×, 30.37×, 29.70×). Once the gather is not the limit, *how* workers are split barely matters; what matters is not spending them on ranks.

Amdahl depends only on W , so it matches the Task 1 row at the same W . Gustafson sits between measurement and Amdahl throughout because its s comes from the parallel run and rises with W .

Comparison against the Week 4 baselines

W	OpenMP	e	Pthreads	e
2	1.96×	0.018	1.57×	0.275
4	3.82×	0.016	2.88×	0.130
8	7.31×	0.014	5.45×	0.067
16	13.39×	0.013	10.21×	0.038

- **OpenMP** has the lowest e in the study (0.013 at $W=16$): nothing is gathered and `schedule(dynamic)` balances the $\sqrt{k}$ cost gradient. One caveat, in OpenMP's disfavour: `week-4-lab-1/task3.c` writes a flag per candidate into an n -byte array and compacts it with an **$O(n)$ serial sweep**, where every other program appends primes directly ($\approx \pi(n)$ writes). It carries serial work the MPI versions never do and still records the lowest e — but the programs are not algorithmically like-for-like.
- **Pthreads'** e starts at 0.275 and *falls* with W — the signature of **load imbalance, not overhead**. Its static block partition gives the thread with the high-numbered candidates far more work, worst when there are only two blocks.

3. Answers to the questions the specification poses

How does the actual speedup compare against the theoretical speedup?

It tracks Amdahl closely to about 32 workers and diverges after: 93% of theory at $W=8$ (7.00× vs 7.53×), 83% at $W=32$ (20.88× vs 25.01×), 58% at $W=64$ (23.87× vs 40.82×). Karp–Flatt localises the divergence — e is flat at ≈ 0.020 while the curve tracks theory and turns up exactly where it stops. So the loss is serial work plus a fixed gather cost to 32 ranks, and communication overhead beyond.

Gustafson sits **below** Amdahl at every W here (37.49× vs 40.82× at $W=64$) because its s comes from the parallel run and grows with W (0.015 $\rightarrow$ 0.42), while Amdahl's stays at 0.0090. For this fixed- n experiment Amdahl is the right model; Gustafson shows what the same code would promise under weak scaling, where the comparison would invert.

Will more MPI processes always increase the speedup?

No — on this hardware the failure is saturation, not collapse.

- In theory Amdahl caps $n=100M$ at 110.9× no matter how many processes.
- In practice the curve rises to 64 ranks but flattens hard: 32 $\rightarrow$ 48 buys 12%, 48 $\rightarrow$ 64 buys 2%. Efficiency: 87% at 8, 77% at 16, 65% at 32, 37% at 64.
- Extrapolating Karp–Flatt, ranks past ~ 80 would start to hurt.
- On a single 10-core laptop the MPI curve *peaked and declined* past the core count — a different regime. Oversubscribed ranks add context-switching to fixed compute and actively hurt; ranks on separate nodes keep helping, by less and less.

How does the workload distribution affect the speedup?

Substantially, and it costs nothing to change. $n=100M$, 16 ranks (sweep C):

scheme	MPI speedup	measured imbalance
blockcyclic	12.35×	1.04×
cyclic	12.34×	1.01×
dynamic	11.54×	1.00×
block	10.04×	4.54×

Imbalance is measured directly as MPI_MAX/MPI_MIN of $t_compute$: 4.54× means the slowest rank computed 4.54× longer than the fastest. (It is in the `imbalance` column of `bench/results-caas/mpi-*.csv`; `analyse.py` does not carry it into `summary.csv`.)

- **block is worst** because trial-division cost grows with the candidate's magnitude, so the rank holding the top block does far more work and the phase ends when it ends. Its e of 0.040 is double `blockcyclic`'s 0.020.
- **This was predicted before it was measured.** Modelling chunk cost as $\propto \sqrt{k}$ gives an expected imbalance of 3.4× (cost $\propto \sqrt{k \ln k}$) to 4.1× (cost $\propto \sqrt{k}$) for **block** against **1.0012×** for `blockcyclic`. Measured: 4.54× and 1.04×. The model slightly understates **block** (it ignores cache effects) but is the right order, and nails `blockcyclic`'s balance. Interleaving balances any smooth, steadily increasing cost function.
- `blockcyclic` and `cyclic` are statistically indistinguishable and recover 23% over **block** on identical hardware.
- **dynamic is the most informative row.** Its imbalance is a perfect **1.00×** — and it is still slower than `blockcyclic`. Balance was never the real limit: there was only 4% left to recover, and it spends a whole rank plus per-chunk request traffic to recover it. Rank 0 idle means 15 of 16 ranks compute, predicting a 1/16 loss; measured gap is 6.6% ($12.35\times \rightarrow 11.54\times$). In the hybrid a reserved rank takes its whole OpenMP team with it: at 4×16 , rank 0 idles **16 of 64 workers**, and `dynamic` measures 21.78× against `blockcyclic`'s 30.33× — a 28% loss against the 25% the lost workers predict. This is why `blockcyclic` is the default: spec-compliant and faster.

Will the speedup results be the same across different machines?

No, and we have direct evidence.

- The Amdahl ceiling is a property of the *program and n* ($s = 0.0090$ at $n=100M$ is the algorithm's phase mix) and transfers between machines. Everything empirical is a property of the host.
- A CAAS core is $\sim 4.7\times$ slower than the laptop core we first measured on (53.7 s vs 11.4 s serial at $n=100M$), yet CAAS reaches 23.9× where the laptop reached under 4×. Absolute speed and scalability are independent.
- Core count sets where the curve turns over; memory bandwidth per core sets how early efficiency decays; the interconnect sets the gather cost that dominates Karp–Flatt past 32 ranks.

4. Reproducing

On the cluster (the numbers in this document):

```
bench/caas.sh push
bench/caas.sh build
bench/caas.sh submit smoke          # wait for SMOKE PASS
bench/caas.sh submit baselines      # serial, pthreads, OpenMP
bench/caas.sh submit mpi            # Task 1
bench/caas.sh submit hybrid         # Task 2
bench/caas.sh pull
bench/caas.sh analyse               # -> bench/analysis-caas/report.html
```

Two traps:

- **Clear stale CSVs out of `bench/results-caas/` before re-analysing** — `analyse.py` globs the whole directory and silently pools runs at different `N_FIXED` into one median.
- **Check `scontrol show partition defq` before trusting any `--time`** — `defq`'s wall limit is 00:20:00, and a job asking for more sits at `PartitionTimeLimit` indefinitely instead of being rejected.

On a single host via Docker (the oversubscription comparison in §3):

```
bench/sweep.sh          # serial, pthreads, OpenMP baselines
bench/sweep-mpi.sh      # Task 1
bench/sweep-hybrid.sh   # Task 2
python3 bench/analyse.py --out bench/analysis
python3 bench/report.py  --out bench/analysis
```

Figures 6 and 7 (`bench/analysis-caas/figures/fig6.svg`, `fig7.svg`) are the two graphs section (c) requires: measured vs Amdahl vs Gustafson, for Task 1 over MPI processes and for Task 2 over ranks × threads.